



LECTURE 16: BINARY SEARCH ON ROTATED & INCOMPLETE DATA

 **Topic: Binary Search** jab data perfect nahi hota

“Real interviews me data clean nahi hota.
Binary Search tab test hota hai jab structure thoda broken ho.”



LECTURE 16 — CORE LEARNING GOAL

Is lecture ka ek hi objective hai:

“Binary Search tab bhi kaam kare, jab array perfect sorted na ho.”

Hum 3 real-world data scenarios solve kar rahe hain:

- 1 Sorted data **rotate** ho gaya
- 2 Rotation ke baad **minimum** dhoondhna
- 3 Sorted data me **numbers missing** hain

 Har problem me:

- brute force possible hai ❌
 - par optimized thinking expected hai ✅
-



PROBLEM 1: SEARCH IN ROTATED SORTED ARRAY

 (LeetCode 33)



PROBLEM STATEMENT (Plain English)

- Ek array pehle sorted tha
- Use kisi unknown index se rotate kar diya gaya
- Sab elements **unique** hain
- Ek **target** diya hai

- Target ka index return karo

arr = [4,5,6,7,0,1,2]

target = 0

Answer = 4

REAL-LIFE INTUITION

Socho:

- Ek dictionary sorted thi
- Kisi ne first few pages ko end me daal diya

Dictionary bigdi nahi —
bas starting point shift ho gaya

👉 Tumhara kaam:

“Kaunsa part abhi bhi sorted hai?”

FIRST-THOUGHT PRINCIPLE (VERY IMPORTANT)

Rotated sorted array ka **strong guarantee**:

✅ **Har moment pe ek half zaroor sorted hota hai**

Binary Search ka kaam:

- pehle sorted half identify karo
- phir check karo target us sorted range me hai ya nahi

👉 Decision > position

MID PAR THINKING (MENTAL FLOW)

At any **mid**, sirf 3 steps:

STEP 1: Target mil gaya?

```
if(arr[mid] == target)
```

STEP 2: Kaunsa half sorted hai?

```
if(arr[mid] >= arr[0]) // LEFT half sorted  
  
else // RIGHT half sorted
```

`arr[0]` yahan **reference anchor** hai
→ original sorted array ka base

STEP 3: Target sorted range me aata hai?

- Aata hai → us side jao
- Nahi aata → doosri side jao

👉 Har step me aadha data delete

💻 CODE (FULLY EXPLAINED)

```
int search(vector<int>& arr, int target) {  
    int left = 0;  
    int right = arr.size() - 1;  
  
    // Jab tak search space valid hai  
    while (left <= right) {  
        // Safe mid calculation (overflow safe)  
        int mid = left + (right - left) / 2;  
  
        // -----  
        // Case 1: Target directly mil gaya  
        // -----  
        if (arr[mid] == target) {  
            return mid;  
        }  
  
        // -----  
        // Case 2: LEFT half sorted hai  
        // Condition:  
        // arr[mid] >= arr[0] means mid left sorted block me hai  
        // -----
```

```

if (arr[mid] >= arr[0]) {
    // Check karo target LEFT sorted range me aata hai ya nahi
    if (target >= arr[left] && target < arr[mid]) {
        // Target definitely left side me hoga
        right = mid - 1;
    } else {
        // Target left range me nahi hai
        left = mid + 1;
    }
}
// -----
// Case 3: RIGHT half sorted hai
// -----
else {
    // Check karo target RIGHT sorted range me aata hai ya nahi
    if (target > arr[mid] && target <= arr[right]) {
        // Target right side me hoga
        left = mid + 1;
    } else {
        // Target right range me nahi hai
        right = mid - 1;
    }
}

// Target array me exist nahi karta
return -1;
}

```



DRY RUN (STEP-BY-STEP WITH REASONING)

arr = [4,5,6,7,0,1,2], target = 0

Iteration 1

left=0, right=6

mid=3 → arr[mid]=7

LEFT half sorted

target >= 4 ? ❌

→ left = 4

Iteration 2

left=4, right=6

mid=5 $\rightarrow$ arr[mid]=1

RIGHT half sorted

target > 1 ? ❌

$\rightarrow$ right = 4

Iteration 3

left=4, right=4

mid=4 $\rightarrow$ arr[mid]=0

FOUND

🕒 TIME & SPACE COMPLEXITY

- Time: $O(\log n)$
 - Har iteration me half array discard
- Space: $O(1)$
 - Sirf variables, no recursion

📌 PROBLEM 2: FIND MINIMUM IN ROTATED SORTED ARRAY

📌 (LeetCode 153)

🧩 PROBLEM STATEMENT

- Sorted array rotate ho gaya hai

- Elements unique hain
- Tumhe **minimum element** dhoondhna hai

```
nums = [10, 20, 30, 40, 50, 60, 70, 1, 2, 3]
```

```
Answer = 1
```



CORE REALIZATION

Tum minimum nahi dhoondh rahe

Tum **rotation break point** dhoondh rahe ho

👉 Minimum wahi hota hai jahan sorted order toot-ta hai



EDGE CASE (INTERVIEW FAVORITE)

```
if(nums[0] <= nums[n-1])
```

Matlab:

- Array already sorted
- No rotation
- Direct answer

👉 Smart engineers **unnecessary binary search avoid karte hain**



DECISION LOGIC (WHY nums[0]?)

`nums[0]` = original sorted order ka reference

- Agar `nums[mid] >= nums[0]`
→ mid abhi left sorted part me
 - Agar `nums[mid] < nums[0]`
→ mid rotated part me (possible minimum)
-

CODE (CLEAN + EXPLAINED)

cpp

```
int findMin(vector<int>& nums) {
    int n = nums.size();

    // -----
    // Edge case: already sorted array
    // -----
    if (nums[0] <= nums[n - 1]) {
        return nums[0];
    }

    int left = 0;
    int right = n - 1;
    int minElement = INT_MAX;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        // -----
        // mid LEFT sorted part me hai
        // minimum RIGHT side me hoga
        // -----
        if (nums[mid] >= nums[0]) {
            left = mid + 1;
        }
        // -----
        // mid ROTATED part me hai
        // mid ek possible minimum ho sakta hai
        // -----
        else {
            minElement = nums[mid];
            right = mid - 1;
        }
    }

    return minElement;
}
```

DRY RUN

nums = [10,20,30,40,50,60,70,1,2,3]

mid=4 → 50 >= 10 → left=5

mid=7 → 1 < 10 → min=1, right=6

mid=5 → 60 >= 10 → left=6

`mid=6 → 70 >= 10 → left=7`

Loop end → `min = 1`

COMPLEXITY

- Time: $O(\log n)$
 - Space: $O(1)$
-

? PROBLEM 3: KTH MISSING POSITIVE NUMBER

 (LeetCode 1539 — Linear Thinking)

PROBLEM KO SAMAJHNE KA SAHI TAREEKA

Natural numbers:

1 2 3 4 5 6 7 8 9 ...

Array:

2 3 4 7 11

Tum mentally pooch rahe ho:

“Ye number array me hai ya missing?”

CORE IDEA (WHY `missingNumber = k`?)

`k` missing numbers milne ke baad
answer **aage shift** ho jaata hai

 Isliye hum `k` ko continuously push karte hain

CODE (Explained)

cpp

```
int findKthPositive(vector<int>& arr, int k) {  
  
    // Ye wo number hai jo hum currently expect kar rahe hain  
  
    int missingNumber = k;  
  
    for (int i = 0; i < arr.size(); i++) {  
  
        // Agar current array element missingNumber se bada hai  
        // matlab missingNumber array me exist hi nahi karta  
  
        if (arr[i] > missingNumber) {  
  
            return missingNumber;  
  
        }  
  
        // Warna ek missing cross ho gaya  
  
        else {  
  
            missingNumber++;  
  
        }  
  
    }  
  
    // Agar array ke end tak pahunch gaye  
  
    return missingNumber;  
  
}
```

DRY RUN

arr=[2,3,4,7,11], k=5

missing=5

2 present → missing=6

3 present → missing=7

4 present → missing=8

7 present → missing=9

11 > 9 → answer = 9

COMPLEXITY

- Time: $O(n)$
- Space: $O(1)$

📌 Binary search optimized version later use hota hai

LECTURE 16 — FINAL THINKING MAP

Problem	Binary Search Used?	Key Question
Search Rotated	YES	Kaunsa half sorted?
Find Minimum	YES	Rotation break kahan?
Kth Missing	NO	Gap kahan aa raha?

FINAL GYAAN (LOCK THIS)

“ Coding = syntax ❌

Coding = structure + decision + elimination ✅